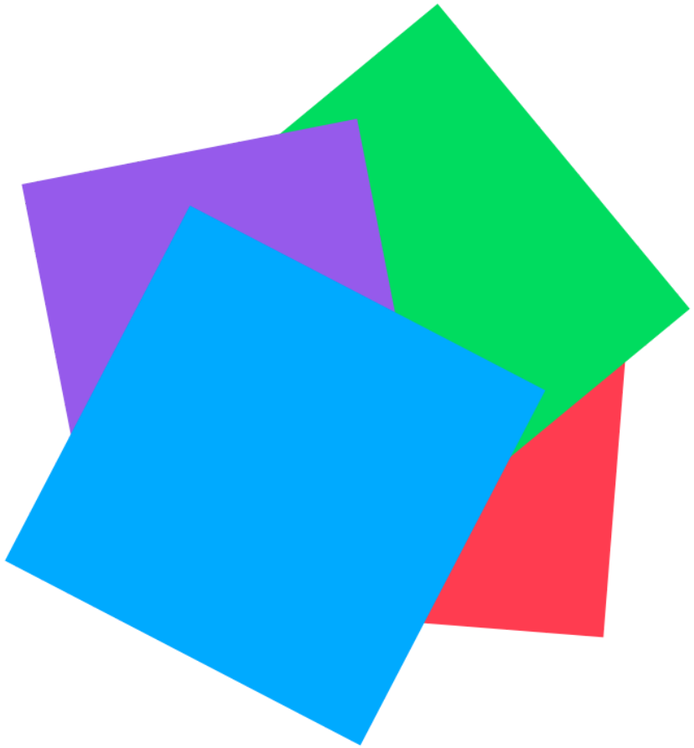
# SDD в существующем проекте

Практика 5 · OpenSpec в brownfield · 90 минут



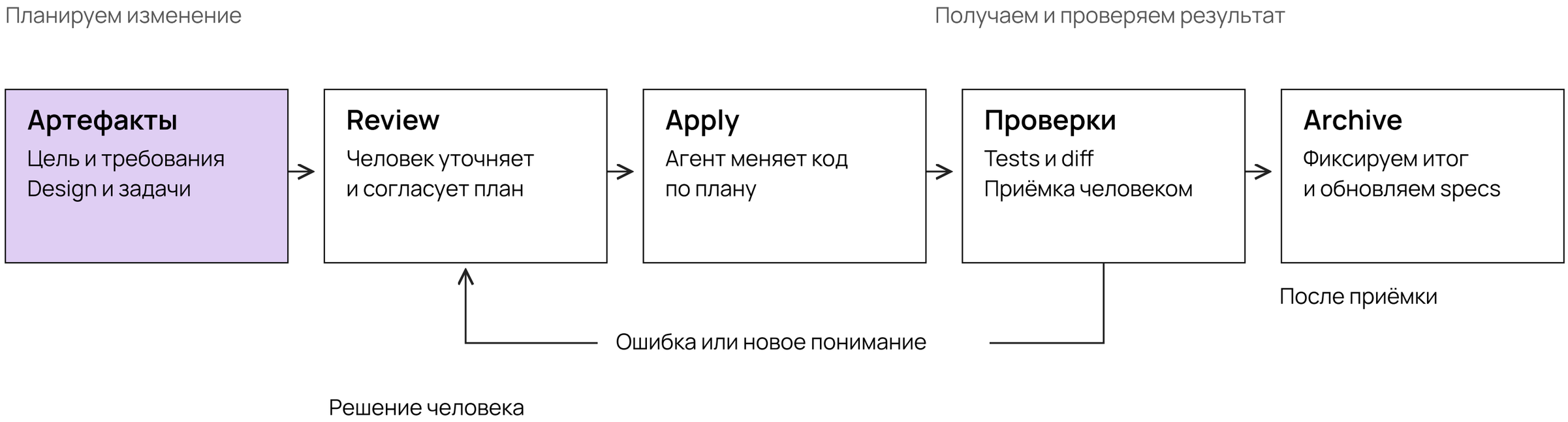
Часть 1: проводим одну фичу через полный цикл.

Часть 2: настраиваем OpenSpec и сравниваем форму до/после.

Ведущий делает → студенты повторяют → проверяем результат.

# Общий цикл OpenSpec

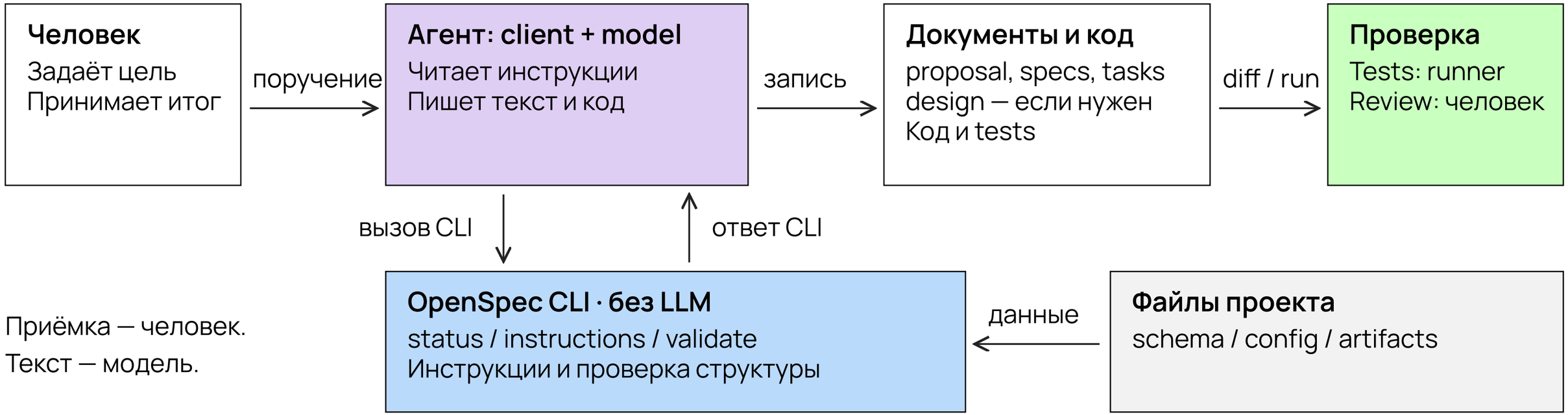
Часть 1 — полный цикл фичи. Часть 2 — настройка формы specs/design.




Сначала делаем намерение и границы изменения видимыми. Артефакты можно уточнять по мере понимания задачи.

# Что делает OpenSpec, а что — модель

CLI собирает инструкции и статус. Содержание изменения создаёт модель в coding client.




Приёмка — человек.
Текст — модель.

Готово: различаем решение человека, генерацию модели и детерминированные операции CLI.



Человек задаёт и принимает поведение; модель пишет; CLI и runner выполняют свои проверки.

# Перед стартом · Устанавливаем OpenSpec

В корне своего проекта. Node.js ≥20.19.0; свои код, правила, tools и tests сохраняем.

Терминал проекта → shell агента

```
node --version
npm install -g @fission-ai/openspec@1.14.1
openspec --version
openspec init --profile core
# Затем попросите агента выполнить:
openspec --version
openspec list --json
```

В init выберите свой coding client; core даёт нужные шесть workflow.

Перезагрузите клиент. В шагах видны OpenCode-команды и реальные имена Codex skills.

Готово: агент видит OpenSpec 1.14.1 и правильный корень вашего проекта.

# Десять небольших фич для своего проекта

Выберите одну полезную идею; новый backend, продукт или framework не требуется.

## Контент и интерфейс

1. Фильтр существующего списка
2. Поиск по имеющемуся контенту
3. Сортировка существующих элементов
4. Сброс фильтров или настроек
5. Копирование ссылки или результата

## Ввод и действия

6. Проверка существующего ввода
7. Экспорт видимых данных в файл
8. Одна локальная настройка
9. Клавиша для существующего действия
10. Предпросмотр существующего изменения

Готово: выбрана одна фича; дальше запишем 3–5 условий и то, что сохраняем.

# Шаг 1. Выбираем фичу, исследуем и обсуждаем

Небольшое улучшение своего проекта: фильтр, сортировка, поле ответа или проверка входа.

OpenCode · Codex: $openspec-explore

```
/opsx-explore
Хочу [небольшая фича моего проекта].
Прочитай затронутый код и существующие тесты.
Что уже есть? Что спорно или лишнее?
Найди команду проверки и запусти baseline.
Предложи узкие границы и вопросы для меня.
Код пока не меняй.
```

Проверьте идею по коду: что уже есть и что можно упростить.

Запишите 3–5 условий приёмки и старое поведение, которое сохраняем.

Готово: выбрана фича; записаны 3–5 условий и то, что должно продолжить работать.

# Шаг 2. Создаём спецификацию

Замените my-feature коротким именем своей фичи.

OpenCode · Codex: $openspec-propose

```
/opsx-propose my-feature
Цель: [одна фича моего проекта].
Используй выводы explore и принятые решения.
Зафиксируй 3-5 условий, ошибки и границы.
Сохрани согласованное прежнее поведение.
Укажи, что не входит в изменение.
Подготовь документы. Остановись до apply.
```

Передайте решения шага 1: цель, условия, ошибки и границы.

Прочитайте предложение до разрешения на код.

Готово: предложение описывает нужное поведение и способ его проверить.

# Как посмотреть созданные specs и changes

CHANGE и SPEC — реальные имена из списка, а не слова для буквального запуска.

CLI · терминал своего проекта

```
openspec list
openspec list --specs
openspec show CHANGE --type change
openspec validate CHANGE --type change \
  --strict --no-interactive
openspec view
```

CHANGE берите из list; SPEC — из list –specs.

Если SPEC есть в списке: openspec show SPEC –type spec.

validate проверяет документы; view — разовый текстовый обзор.

Готово: выбран реальный change. Пустой список main specs до первого archive — нормально.

# Шаг 3. Проверяем и уточняем решение

Сначала отдельный проверяющий чат или агент. Затем решение человека.

Проверяющий чат → основной чат OpenCode

```
# Отдельный чат / агент-ревьюер
Прочитай предложение my-feature и связанный код.
Проверь цель, допущения, ошибки и совместимость.
Покажи конкретные пробелы. Файлы не меняй.
# Если нужно уточнение: основной чат OpenCode
/opsx-update my-feature
Учти согласованное решение: [уточнение].
```

- Разберите замечания; не требуйте выдуманных ошибок.
- В Codex уточнение вызывает $openspec-update-change my-feature.

Готово: вопросы разобраны; вы прочитали итог и разрешили apply.

# Шаг 4. Реализуем фичу

Запускайте apply после принятия спецификации.

OpenCode · Codex: $openspec-apply-change

```
/opsx-apply my-feature
Реализуй согласованное изменение.
Используй архитектуру и проверки проекта.
Не расширяй границы задачи.
При новом решении сначала обсуди его со мной.
Покажи изменения и результаты проверок.
Остановись до archive.
```

Следите за границами задачи. Если spec неверна или появились новые факты — остановите код и согласуйте правку.

Обновите документы: `/opsx-update my-feature` (Codex: `$openspec-update-change my-feature`). Прочитайте и примите diff; затем apply и проверки.

Готово: есть код, просмотренные изменения и фактические результаты проверок.

# Шаг 5. Проверяем и принимаем результат

Используйте настоящую команду проверки своего проекта.

Отдельный проверяющий чат / агент

```
Прочитай требования my-feature и изменения.
Найди и запусти команду проверки этого проекта.
Проверь основной, ошибочный и старый сценарии.
По каждому: ожидание → факт → доказательство.
Файлы не меняй.
Не считай отчёт автора самостоятельной проверкой.
```

- Разберите результат независимой проверки и исправьте существенное.
- Человек сопоставляет условия и факт; к шагу 6 переходим после приёмки.

Готово: условия проверены; вы можете объяснить, почему принимаете фичу.

# Шаг 6. Обновляем specs и архивируем

Только после приёмки человеком.

OpenSpec · чат OpenCode; имена Codex справа

```
/opsx-sync my-feature
Покажи diff текущей спецификации.
Сверь её с принятым поведением.

/opsx-archive my-feature
Покажи main specs и запись в archive.
```

Проверьте, что main spec описывает принятое поведение.

В Codex: $openspec-sync-specs, затем $openspec-archive-change.

Готово: текущая спецификация актуальна, завершённое изменение сохранено в истории.

Часть 2

# Часть 2. Настраиваем OpenSpec под себя

**77–90 мин** · Один раз задаём правило проекта; проверяем его на одном фрагменте.

## Сейчас

1. Сохраним фрагмент «до» и постоянное правило проекта.
2. Проверим действие правила на этом фрагменте.

## Готово, когда

- ☐ Правило остаётся в config для следующих изменений.
- ☐ Diff принят человеком; смысл фрагмента сохранён.

Шаг 7 — исходный фрагмент и правило. Шаг 8 — преобразование и проверка.

# Шаг 7. Добавляем правило OpenSpec для проекта

Один раз сохраняем правило проекта; OpenSpec включает его в будущие инструкции.

OpenCode · Codex: $openspec-propose

```
/opsx-propose my-next-feature
Цель: [связанная фича]. Остановись до apply.
# Сохраните существующий фрагмент «до»
# Объедините с текущим openspec/config.yaml:
rules:
  design:
    - Объясни взаимодействие диаграммой Mermaid.
```

- Дополните текущий rules; второй rules не создаём.
- Правило действует для будущих design проекта; design нужен не всегда.
- Если design нет: rules.specs и Given/When/Then для существующей spec.

Готово: правило сохранено для будущих design; фрагмент «до» нужен для проверки.

# Шаг 8. Меняем форму и проверяем смысл

Сначала свежие инструкции, затем предлагаемый diff и подтверждение человека.

OpenCode · Codex: $openspec-update-change

```
/opsx-update my-next-feature
Перед правкой выполни в shell:
openspec instructions design --change my-next-
feature --json
Проверь в ответе моё актуальное правило.
Примени его к тому же фрагменту; покажи diff.
Сохрани смысл и границы; правь после подтверждения.
Код не меняй.
```

- Для spec: в CLI укажите specs, в config — rules.specs; назовите свой файл.
- После подтверждения сравните до/после; Mermaid откройте как диаграмму.

Готово: правило проверено на одном draft и остаётся для следующих изменений проекта.

# ДЗ 5 · Две команды — одна фича

Выберите другой SDD-фреймворк и согласуйте две спецификации до генерации кода.

### Эмуляция работы двух команд · два чата

**КОМАНДА 1**

**Спека UI**

Экраны и действия.
Состояния и ошибки.

**КОМАНДА 2**

**Спека логики**

Данные и алгоритм.
Правила и API.

**Согласуйте общий контракт**

Сверьте данные, вызовы, ошибки и состояния.
Примите решения и обновите обе спеки.

**После согласования:** реализация →
сквозной сценарий и проверка ошибки / границы.

| За что | Баллы |
|---|---|
| **Другой SDD-фреймворк**<br>Применение и сравнение с OpenSpec | 2 |
| **Две спецификации**<br>UI и логика, согласованные до кода | 2 |
| **Реализация**<br>Работающие UI и логика, сквозные проверки | 5 |
| **Рефлексия**<br>Что согласовали и чему научились | 1 |

Сдача: спеки до/после, общий контракт, код, проверки и короткое демо.